Class Name	Responsibilities	Collaborators	Page
Person	• Store common personal data (name, contact info) • Provide access to personal details • Allow updating of contact information • Serve as a base class for all person type classes	**Staff**, **Passenger**	**2**
Staff	• Store staff-specific info (staffID, role) • Provide access to staff details • Allow staff to view their work schedule	**Person** (inherits from), **Schedule** (used in *viewSchedule()*)	**3**
Passenger	• Store passenger-specific info (passengerID, passportNumber) • Provide access to passenger details • Allow passengers to view their tickets	**Person** (inherits from), **Ticket** (used in *viewTickets()*)	**4-5**
StaffAssignment	• Keep track of a specific staff assignment to a flight • Manage the list of crew members assigned to the flight • Allow adding/removing crew members • Provide access to the flight and current crew members	**Staff** (crew members), **Flight** (flight being assigned)	**6-7**
StaffManager	• Maintain a collection of all staff members • Add, remove, or find staff members • Assign staff to flights • Retrieve staff assigned to a given flight • Manager for staff stuff	**Staff** (staff being managed), **Flight** (for assignments)	**8-9**
FlightAssignment	• Link a flight to a specific aircraft • Allow changing the aircraft assigned to the flight • Provide access to flight and aircraft information	**Flight** (flight being assigned), **Aircraft** (aircraft being used)	**10**
****Yellow Highlights means that suggested adjustments are required to make it work**			

Person

Instance Variables:
name: String Stores person's full name
contactInfo: String Stores email and phone number

Methods:

Person(String name, String contactInfo)
● Constructor to initialize the person's name and contact info
● Called by subclasses (**Staff** or **Passengers**) using ***super(name, contactInfo)***
 	
`public Person(String name, String contactInfo){`
`  this.name = name;`
`  this.contactInfo = contactInfo;`
`}`

getName(): String
● Returns the person's name
● Displaying name on tickets, schedules, or profiles
 	
`public String getName() {`
`  return name;`
`}`

getContactInfo(): String
● Returns the contact info (email/phone)
● Retrieving contact info of the person, would be used for sending alerts or flight notifications
 	
`public String getContactInfo(){`
`  return contactInfo;`
`}`

updateContactInfo(String newInfo): void
● Update a person's contact info
● Passenger changes phone/email and staff update their work contact info or personal contact
 	
`public void updateContactInfo(String newInfo) {`
`  this.contactInfo = newInfo;`
`}`

Staff

Instance Variables:

staffID: String
Unique identifier for each staff member (makes ease for StaffAssignment)

role: String
Staff role (Pilot, Cabin Crew, etc)

Methods:

Staff(String name, String contactInfo, String staffID, String role)

- Constructor to initialize both person and staff-specific info
- Creating new staff objects in the system

```
public Staff(String name, String contactInfo, String staffID, String role) {
    super(name, contactInfo);
    this.staffID = staffID;
    this.role = role;
}
```

viewSchedule(): Schedule

- Should create a new schedule object in the format (String, date) like the Constructor shows in class Schedule
- ***manager.getAllAssignments()*** returns all **StaffAssignment** objects
- ***for (StaffAssignment a : assignments)*** loops over all the assignment objects
- ***a.getCrewMembers().contains(this)*** checks if this staff is in the crew for that flight
- if so, we ***mySchedule.addFlight(a.getFlight())*** which adds it to our new schedule for storing

```
public Schedule viewSchedule(StaffManager manager) {
    Schedule mySchedule = new Schedule("SCHEDULE-" + this.getStaffID(), new Date());

    List<StaffAssignment> assignments = manager.getAllAssignments();

    for (StaffAssignment a : assignments) {
        if (a.getCrewMembers().contains(this)) {  // check if staff member is on the flight
            mySchedule.addFlight(a.getFlight());   // add flight from StaffAssignment to schedule
        }
    }
    return mySchedule;
}
```

getStaffID(): String

- Returns the staff ID
- Identifying staff in assignments or payroll

```
public String getStaffID() {
    return staffID;
}
```

getRole(): String

- Return staff role
- Checking permissions (their role e.g. Pilot, Cabin Crew, etc)

```
public String getRole() {
    return role;
}
```

Passenger

Instance Variables:
passengerID: String Unique identifier for each passenger
passportNumber: String Passport number for identification

Methods:

Passenger(String name, String contactInfo, String passengerID, String passportNumber)
• Constructor to initialize both person and passenger-specific info
• Creating a passenger profile in the system
 	
`public Passenger(String name, String contactInfo, String passengerID, String passportNumber) {`
`  super(name, contactInfo);`
`  this.passengerID = passengerID;`
`  this.passportNumber = passportNumber;`
`}`

viewTickets(): ArrayList<Ticket>
• Grab list of all tickets from TicketManager when calling viewTickets
public ArrayList<Ticket> viewTickets(TicketManager manager)
• Create new empty list called ***myTickets*** to store that passenger's tickets
• Call a function to get ALL tickets stored in TicketManager ***manager.getAllTickets()***
• ***for (Ticket t : manager.getAllTickets())*** loops through each ticket (***t***) in that list
• use an if statement to check if the ticket belongs to the passenger ***t.getPassenger().equals(this)***, if it matches, add it to ***myTickets*** and return
 	
• This also means that class Ticket should have a way to see what passenger holds it, e.g. ***public Passenger getPassenger()*** (we don't have any other way to reference the passenger when grabbing the ticket info so this would help when viewing their owned tickets)
 	
`public ArrayList<Ticket> viewTickets(TicketManager manager) {`
`  ArrayList<Ticket> myTickets = new ArrayList<>();`
`  for (Ticket t : manager.getAllTickets()){`
`  if (t.getPassenger().equals(this)){`
`  myTickets.add(t);`
`  }`
`  }`
`  return myTickets;`
`}`
• Summary: need ***getAllTickets()*** to be added to **TicketManager** and ***getPassenger()*** to be added to **Ticket**
• Core algorithm should work in viewTickets method if those are added

updateContactInfo(String newInfo): void • Optional override to add extra passenger-specific behavior when updating contact info • Passenger changes email/phone; system can trigger notifications automatically

```java
@Override
public void updateContactInfo(String newInfo) {
    super.updateContactInfo(newInfo);
}
```

getPassengerID(): String

- Return passenger ID
- Booking, check-in, loyalty programs

```java
public String getPassengerID() {
    return passengerID;
}
```

getPassportNumber(): String

- Return passport number
- If needed for international flights (maybe just would need hasPassport())

```java
public String getPassportNumber() {
    return passportNumber;
}
```

StaffAssignment

Instance Variables:
assignmentID: String Unique identifier for the assignment
flight: Flight The flight that this assignment belongs to
crewMembers: ArrayList<Staff> This list stores all the staff assigned to this flight
allAssignments: ArrayList<StaffAssignments> Stores all assignments in the system and is needed so other classes can access every assignment in one place

Methods:

StaffAssignment(String assignmentID, Flight flight)
● Creates a new assignment
● Initializes crew list
● Adds itself to the global list
 	
`public StaffAssignment(String assignmentID, Flight flight) {`
`  this.assignmentID = assignmentID;`
`  this.flight = flight;`
`  this.crewMembers = new ArrayList<>();`
 	
`  allAssignments.add(this); //register this specific assignment`
`}`

assignCrewMember(Staff member): void
● Adds a staff member to the crew list
 	
`public void assignCrewMember(Staff member) {`
`  crewMembers.add(member);`
`}`

removeCrewMember(Staff member): void
● Removes a staff member from the crew
 	
`public void removeCrewMember(Staff member) {`
`  crewMembers.remove(member);`
`}`

getCrewMembers(): List<Staff>

- Returns all crew members for this flight

```
public List<Staff> getCrewMembers() {
    return crewMembers;
}
```

getFlight(): Flight

- Returns the associated flight

```
public Flight getFlight() {
    return flight;
}
```

getAllAssignments(): List<StaffAssignment>

- Returns all assignments in the system that have been created in this class

```
public static List<StaffAssignment> getAllAssignments() {
    return allAssignments;
}
```

StaffManager

Instance Variables:
staffMembers: ArrayList<Staff> List of employees in the airline system, ex: Pilots, co-pilots, flight attendants- all of them

Methods:

StaffManager()

- Constructor that starts the staff list empty
- This initializes the list so we can start making changes

```
public StaffManager() {
    staffMembers = new ArrayList<>();
}
```

addStaff(Staff s): void

- Adds a staff member to the list

```
public void addStaff(Staff s) {
    staffMembers.add(s);
}
```

removeStaff(String staffID): void

- Removes a staff member by ID number
- Looks at each staff in list one by one
- Compares their ID to the one you want to remove
- If it matches, remove it

```
public void removeStaff(String staffID) {
    Iterator<Staff> iterator = staffMembers.iterator();
    while (iterator.hasNext()) {
        Staff s = iterator.next();
        if (s.getStaffID().equals(staffID)) {
            iterator.remove(); // safe removal
        }
    }
}
```

findStaff(String staffID): Staff

- Searches for a staff member by ID
- Returns the staff member object if found, otherwise returns null
- Looks at each staff, if the ID matches, return the staff

```
public Staff findStaff(String staffID) {
    for (Staff s : staffMembers) {
        if (s.getStaffID().equals(staffID)) {
            return s;
        }
    }
    return null;
}
```

assignStaffToFlight(Staff s, Flight f): void

- Links a staff member to a flight
- Checks if this flight already has an existing assignment
- If it does, add staff to the crew list
- It it doesn't have an existing assignment, make a new one

```
public void assignStaffToFlight(Staff s, Flight f) {
    //find existing assignment for this flight
    for (StaffAssignment sa : StaffAssignment.getAllAssignments()) {
        if (sa.getFlight().equals(f)) {
            sa.assignCrewMember(s);
            return;
        }
    }
    //if no assignment exists, create one
    String newAssignmentID = "ASSIGN-" + f.getFlightNumber() + "-" + s.getStaffID();
    StaffAssignment newAssignment = new StaffAssignment(newAssignmentID, f);
    newAssignment.assignCrewMember(s);
}
```

- In order for the second block to work, we need a getter for the **flightNumber** attribute in Flight

getStaffForFlight(Flight f): List<Staff>

- Returns all of the staff assigned to a flight
- Looks at all assignments
- Finds the one for the specific flight
- Returns the list of staff members
- If nothing exists, return an empty list

```
public List<Staff> getStaffForFlight(Flight f) {
    for (StaffAssignment sa : StaffAssignment.getAllAssignments()) {
        if (sa.getFlight().equals(f)) {
            return sa.getCrewMembers();
        }
    }
    return new ArrayList<>();
}
```

getAllAssignments(): List<StaffAssignments>

- Returns all assignments in the system, which is needed so staff can check their schedule

```
// needed for Staff.viewSchedule()
public List<StaffAssignment> getAllAssignments() {
    return StaffAssignment.getAllAssignments();
}
```

FlightAssignment

Instance Variables:
assignmentID Unique identifier for the assignment
flight: Flight The flight this object is referring to
aircraft: Aircraft The aircraft assigned to the flight

Methods:

FlightAssignment(String assignmentID, Flight flight, Aircraft aircraft)
● Constructor to initialize all fields
 	
`public FlightAssignment(String assignmentID, Flight flight, Aircraft aircraft) {`
`  this.assignmentID = assignmentID;`
`  this.flight = flight;`
`  this.aircraft = aircraft;`
`}`

getFlight(): Flight
● Returns the flight for reference
 	
`public Flight getFlight() {`
`  return flight;`
`}`

getAircraft(): Aircraft
● Returns the aircraft used for the flight
 	
`public void removeCrewMember(Staff member) {`
`  crewMembers.remove(member);`
`}`

changeAircraft(Aircraft newAircraft): void
● Updates the aircraft
 	
`public void changeAircraft(Aircraft newAircraft) {`
`  this.aircraft = newAircraft;`
`}`